

UNIVERSIDAD AUTÓNOMA METROPOLITANA

Sistema P2P para Compartición de Video

con Broker Coordinador y Control de Estados

Sistemas Distribuidos

Autores:

Aaron Rodrigo Ramos Reyes

Ismael Valencia Reyes

Fabian Emiliano Castañeda Juvera

21 de julio de 2026

Resumen

Este documento describe el diseño e implementación de un sistema Peer-to-Peer (P2P) para la compartición de archivos de video. El sistema utiliza un broker central que actúa únicamente como coordinador, manteniendo una lista de los peers activos y notificando a todos cuando un nuevo peer se une o se retira. Cada peer puede actuar simultáneamente como cliente y servidor: puede descargar videos de otros peers mientras otros descargan de él. Los peers están implementados en C++ con un backend que maneja las conexiones y un frontend gráfico en Python que muestra la lista de peers, sus estados (IDLE, UPLOADING, DOWNLOADING) y una barra de progreso durante las descargas. El sistema es totalmente descentralizado en las transferencias, ya que los archivos se transmiten directamente entre peers sin pasar por el broker. Este reporte explica en detalle la arquitectura, el protocolo de comunicación, el manejo de estados, el flujo de datos y cómo levantar y usar el sistema.

Índice

1. Introducción	3
1.1. ¿Qué es un sistema P2P?	3
1.2. ¿Por qué usar un broker?	3
1.3. Objetivos del sistema	3
2. Arquitectura del Sistema	3
2.1. Componentes principales	3
2.2. Flujo de comunicación	4
3. Protocolo de Comunicación	4
3.1. Comandos entre Peer y Broker	4
3.2. Comandos entre Peers (P2P)	5
3.3. Comandos entre Peer Backend y Frontend	5
4. Manejo de Estados de los Peers	6
4.1. ¿Cómo se actualizan los estados?	6
4.2. Ejemplo de estado en el frontend	6
5. Flujo de Datos para una Descarga	6
6. Implementación del Código	7
6.1. Estructura del Peer Backend (C++)	7
6.1.1. Conexión al Broker	8
6.1.2. Servidor P2P (para atender otros peers)	8
6.1.3. Manejo de Descargas	8
6.2. Frontend Python (Interfaz Gráfica)	9
6.2.1. Recepción de mensajes del backend	9

7. Levantamiento y Uso del Sistema	10
7.1. Requisitos previos	10
7.2. Paso a paso	10
7.2.1. 1. Compilar el broker	10
7.2.2. 2. Compilar el peer backend	10
7.2.3. 3. Ejecutar el broker	10
7.2.4. 4. Ejecutar un peer (en cada máquina)	10
7.2.5. 5. Ejecutar el frontend (en cada máquina)	10
7.2.6. 6. Usar el sistema	11
7.3. Ejemplo de ejecución con 3 peers	11
8. Pruebas y Resultados	11
9. Conclusión	12
10.Evidencias de Funcionamiento	12
11.Entorno de Prueba	12
12.Registro de Peers en el Broker	13
13.Frontend: Lista de Peers y Estados	13
14.Carga y Descarga Simultánea	14
14.1. Prueba 1: Descarga mutua	14
14.2. Prueba 2: Un peer sirviendo a múltiples clientes	14
15.Logs de las Terminales	15
15.1. Logs del Broker	15
15.2. Logs del Peer que Descarga	15
15.3. Logs del Peer que Sube	15
16.Validación de Estados en el Frontend	15
17.Conclusión de las Pruebas	16

1. Introducción

1.1. ¿Qué es un sistema P2P?

Un sistema Peer-to-Peer (P2P) es una red donde cada nodo (llamado "peer") actúa tanto como cliente (solicita recursos) como servidor (ofrece recursos). A diferencia de los modelos cliente-servidor tradicionales, no hay un servidor central que almacene todos los archivos; los archivos se distribuyen entre los peers y se transfieren directamente entre ellos.

1.2. ¿Por qué usar un broker?

En una red P2P pura, cada peer necesitaría conocer la dirección de todos los demás, lo que es difícil de mantener cuando la red cambia constantemente (peers entran y salen). El broker es un servidor ligero que solo mantiene una lista actualizada de los peers activos. Cuando un peer quiere unirse, se registra en el broker, y el broker notifica a todos los demás. Luego, las transferencias de archivos ocurren directamente entre peers, sin pasar por el broker. Esto mantiene el tráfico descentralizado y escalable.

1.3. Objetivos del sistema

- Permitir que múltiples peers compartan archivos de video en una red local.
- Cada peer debe poder descargar y subir archivos simultáneamente.
- El broker debe mantener la lista de peers activos y notificar cambios.
- El frontend debe mostrar la lista de peers, su estado (si están subiendo o descargando) y una barra de progreso durante las descargas.
- La transferencia de archivos debe ser directa entre peers (sin pasar por el broker).

2. Arquitectura del Sistema

2.1. Componentes principales

El sistema consta de tres tipos de componentes:

Componente	Lenguaje	Función
Broker	C++	Mantiene lista de peers, notifica cambios. No transfiere archivos.
Peer Backend	C++	Se conecta al broker, corre servidor P2P, maneja descargas, comunica estados al frontend.
Peer Frontend	Python	Interfaz gráfica: muestra peers con estados, lista archivos, barra de progreso.

Cuadro 1: Componentes del sistema P2P

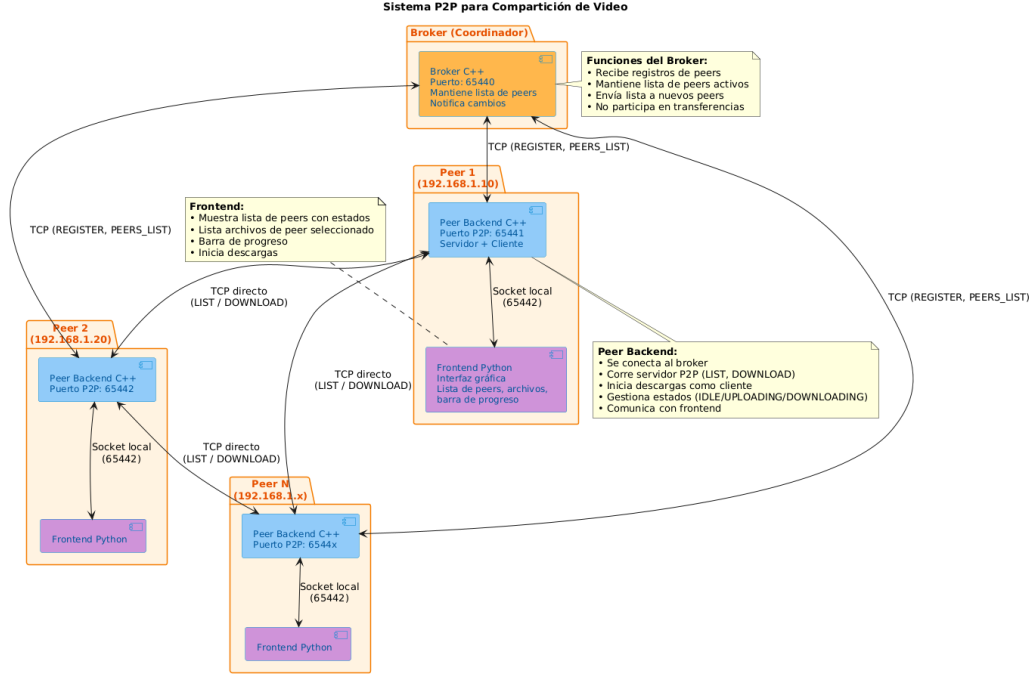


Figura 1: Arquitectura del sistema P2P con broker

2.2. Flujo de comunicación

1. El broker se inicia y escucha en el puerto 65440.
2. Cada peer backend se conecta al broker y se registra (envía su IP y puerto P2P).
3. El broker añade el peer a su lista y envía la lista completa a todos los peers.
4. Los peers actualizan su lista local y la envían a sus frontends.
5. Cuando un peer solicita una descarga, se conecta directamente al peer destino (usando su IP y puerto P2P) y negocia la transferencia sin intervención del broker.
6. Durante la transferencia, los peers cambian su estado (UPLOADING o DOWNLOADING) y el frontend lo muestra.

3. Protocolo de Comunicación

3.1. Comandos entre Peer y Broker

Comando	Descripción
REGISTER ip puerto	El peer se registra en el broker.
GET_PEERS	El peer solicita la lista actual de peers.
PEERS_LIST	El broker envía la lista de todos los peers (separados por espacio).
ip1:puerto1 ip2:puerto2 ...	

Cuadro 2: Comandos entre peer y broker

3.2. Comandos entre Peers (P2P)

Comando	Descripción
LIST	Solicita la lista de archivos disponibles en el peer remoto.
DOWNLOAD nombre	Solicita la descarga de un archivo. El peer remoto responde con SIZE tamaño seguido de los datos binarios.

Cuadro 3: Comandos entre peers

3.3. Comandos entre Peer Backend y Frontend

Comando	Descripción
GET_PEERS	El frontend solicita la lista de peers al backend.
PEERS_LIST ip1:puerto1 ip2:puerto2 ...	El backend envía la lista de peers al frontend.
PEER_STATUS ip:puerto ESTADO	El backend notifica un cambio de estado de un peer.
LIST_FROM_PEER ip puerto	El frontend pide listar archivos de un peer remoto.
DOWNLOAD_FROM_PEER ip puerto archivo	El frontend inicia una descarga.
PROGRESS 45	El backend envía el progreso de la descarga.
DOWNLOAD_COMPLETED	Se notifica que la descarga terminó.

Cuadro 4: Comandos entre backend y frontend

4. Manejo de Estados de los Peers

Cada peer puede estar en uno de tres estados:

- **IDLE**: No está realizando ninguna transferencia.
- **UPLOADING**: Está enviando un archivo a otro peer (actúa como servidor).
- **DOWNLOADING**: Está recibiendo un archivo de otro peer (actúa como cliente).

El backend del peer notifica al frontend cada vez que su estado cambia, y el frontend actualiza la lista de peers mostrando el estado entre paréntesis. Esto permite a los usuarios ver qué peers están activos y qué actividad están realizando.

4.1. ¿Cómo se actualizan los estados?

1. Cuando un peer inicia una descarga, el backend cambia su estado a **DOWNLOADING** y envía **PEER_STATUS** a su frontend.
2. Cuando un peer recibe una solicitud de descarga de otro peer, cambia su estado a **UPLOADING** y lo notifica a su frontend.
3. Al terminar la transferencia, ambos vuelven a **IDLE**.

4.2. Ejemplo de estado en el frontend

Peers disponibles (estado)

192.168.1.10:65441 (IDLE)

192.168.1.20:65442 (UPLOADING) # Está subiendo un archivo

192.168.1.30:65443 (DOWNLOADING) # Está descargando un archivo

5. Flujo de Datos para una Descarga

El siguiente diagrama de secuencia muestra cómo ocurre una descarga entre dos peers:

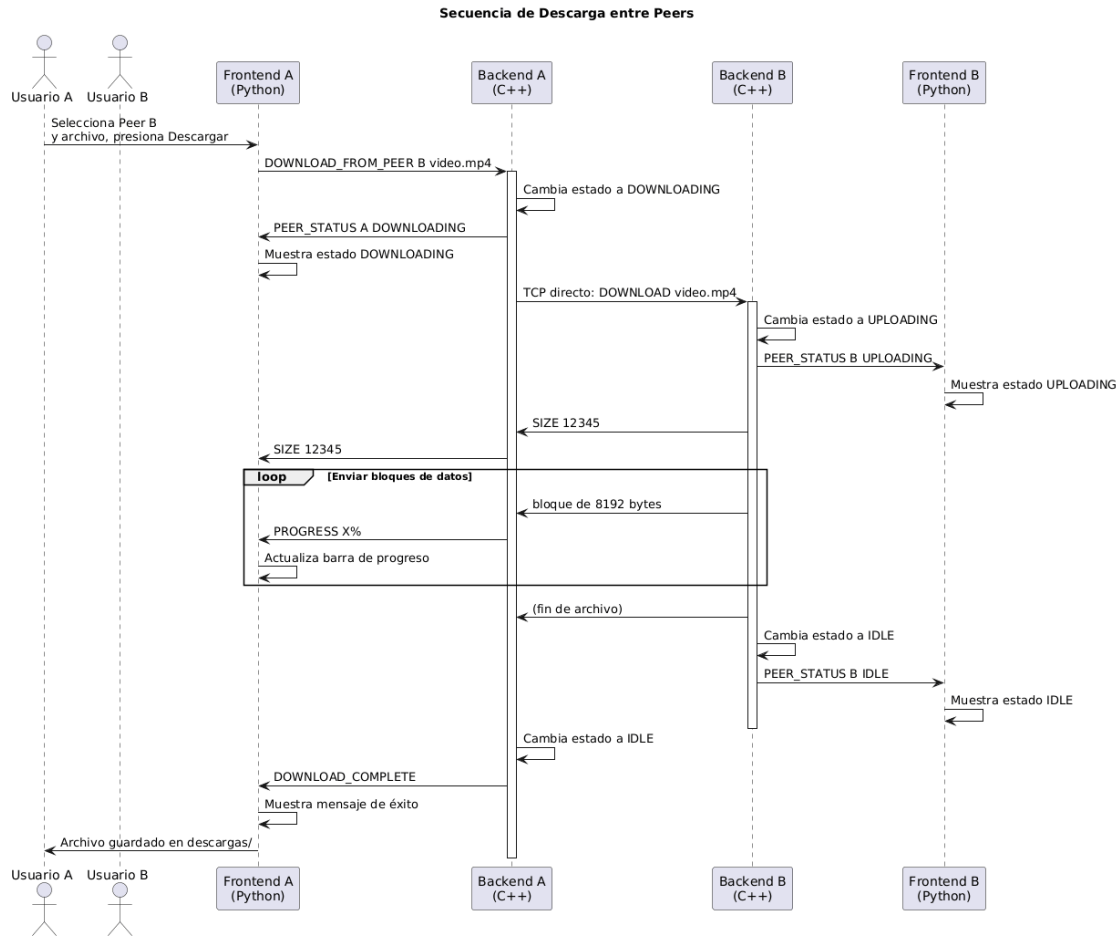


Figura 2: Secuencia de descarga entre dos peers

Pasos: 1. El frontend del Peer A envía `DOWNLOAD_FROM_PEER` al backend de A. 2. El backend de A cambia su estado a `DOWNLOADING` y lo notifica a su frontend. 3. El backend de A se conecta directamente al Peer B (usando IP:puerto P2P) y envía `DOWNLOAD` archivo. 4. El Peer B recibe la solicitud, cambia su estado a `UPLOADING` y lo notifica a su frontend. 5. El Peer B envía `SIZE` tamaño a A. 6. El Peer B envía los datos del archivo en bloques. 7. A medida que A recibe datos, calcula el progreso y lo envía a su frontend (mensajes `PROGRESS`). 8. Cuando A ha recibido todos los datos, envía `DOWNLOAD_COMPLETE` a su frontend. 9. Ambos peers vuelven a `IDLE`.

6. Implementación del Código

6.1. Estructura del Peer Backend (C++)

El peer backend es el núcleo del sistema. Se encarga de:

1. Conectarse al broker y registrarse.
2. Mantener la lista de peers actualizada.

3. Ejecutar un servidor P2P para atender solicitudes de otros peers (LIST y DOWNLOAD).
4. Actuar como cliente para descargar archivos de otros peers.
5. Comunicarse con el frontend mediante un socket local.
6. Gestionar los estados y notificarlos al frontend.

6.1.1. Conexión al Broker

```
1 bool conectar_broker(const std::string& broker_ip) {
2     broker_fd = socket(AF_INET, SOCK_STREAM, 0);
3     // ... configurar direccion ...
4     connect(broker_fd, (struct sockaddr*)&addr, sizeof(addr));
5     // Enviar registro
6     std::string reg = "REGISTER " + mi_ip + " " + std::to_string(
7     mi_puerto_p2p) + "\n";
8     send(broker_fd, reg.c_str(), reg.size(), 0);
9     // Iniciar hilo para escuchar mensajes del broker
10    std::thread(escuchar_broker).detach();
11 }
```

Listing 1: Conexión del peer al broker

Explicación: El peer crea un socket TCP y se conecta al broker. Luego envía un mensaje REGISTER con su IP y puerto P2P. A partir de ese momento, escucha en un hilo separado los mensajes que el broker le envíe (como PEERS_LIST o notificaciones de nuevos peers).

6.1.2. Servidor P2P (para atender otros peers)

```
1 void iniciar_servidor_p2p() {
2     p2p_server_fd = socket(AF_INET, SOCK_STREAM, 0);
3     // ... bind a mi_puerto_p2p ...
4     listen(p2p_server_fd, 5);
5     while (true) {
6         client_fd = accept(...);
7         std::thread(atender_peer_p2p, client_fd).detach();
8     }
9 }
```

Listing 2: Servidor P2P del peer

Explicación: El peer abre un puerto P2P (ej. 65441) y se queda escuchando. Por cada conexión entrante, crea un hilo para atenderla. Esto permite que múltiples peers descarguen de él simultáneamente.

6.1.3. Manejo de Descargas

Cuando el peer recibe un comando DOWNLOAD de otro peer:

```

1 void manejar_descarga(int client_fd, const std::string& filename, const
  std::string& peer_key) {
2     notificar_estado(peer_key, "UPLOADING");    // Cambiar a subiendo
3     // Abrir archivo, enviar SIZE y datos...
4     notificar_estado(peer_key, "IDLE");        // Volver a idle al
  terminar
5 }

```

Listing 3: Manejo de descarga en el servidor P2P

Cuando el peer inicia una descarga como cliente:

```

1 if (comando == "DOWNLOAD_FROM_PEER") {
2     notificar_estado(mi_key, "DOWNLOADING");    // Cambiar a descargando
3     // Conectar al peer remoto, enviar DOWNLOAD, recibir archivo...
4     notificar_estado(mi_key, "IDLE");        // Volver a idle al
  terminar
5 }

```

Listing 4: Inicio de descarga como cliente

6.2. Frontend Python (Interfaz Gráfica)

El frontend usa Tkinter y se comunica con el backend a través de un socket local. Muestra:

- Una lista de peers con sus estados (actualizados en tiempo real).
- Una lista de archivos del peer seleccionado.
- Una barra de progreso para las descargas.
- Un campo para escribir el nombre del archivo a descargar.

6.2.1. Recepción de mensajes del backend

```

1 def process_message(self, msg):
2     if msg.startswith("PEERS_LIST"):
3         # Actualizar lista de peers
4         self.peer_listbox.delete(0, tk.END)
5         for part in parts:
6             self.peer_listbox.insert(tk.END, f"{key} ({estado})")
7     elif msg.startswith("PEER_STATUS"):
8         # Actualizar estado de un peer
9         self.peer_states[key] = estado
10        self.update_peer_list()
11    elif msg.startswith("PROGRESS"):
12        # Actualizar barra de progreso
13        self.progress_var.set(percent)
14    elif msg.startswith("DOWNLOAD_COMPLETE"):
15        # Mostrar mensaje de exito
16        messagebox.showinfo("Exito", "Descarga completada")

```

Listing 5: Procesamiento de mensajes en el frontend

7. Levantamiento y Uso del Sistema

7.1. Requisitos previos

- Compilador g++ con soporte C++17 y pthreads.
- Python 3 con Tkinter instalado.
- Las máquinas deben estar en la misma red (o con puertos accesibles).

7.2. Paso a paso

7.2.1. 1. Compilar el broker

En la máquina que actuará como coordinador:

```
1 g++ -std=c++17 -pthread broker.cpp -o broker
```

7.2.2. 2. Compilar el peer backend

En cada máquina que será un peer:

```
1 g++ -std=c++17 -pthread peer_backend.cpp -o peer
```

7.2.3. 3. Ejecutar el broker

```
1 ./broker
```

Verás: Broker P2P - Coordinador de Peers / Escuchando en puerto 65440

7.2.4. 4. Ejecutar un peer (en cada máquina)

```
1 ./peer <broker_ip> <mi_ip> <puerto_p2p> [directorio_videos]
```

Ejemplo (en la máquina con IP 192.168.1.10, broker en 192.168.1.1):

```
1 ./peer 192.168.1.1 192.168.1.10 65441 videos/
```

Importante: Cada peer debe usar un puerto P2P diferente (ej. 65441, 65442, 65443). Si todos están en la misma máquina, usen IP 127.0.0.1 y puertos distintos.

7.2.5. 5. Ejecutar el frontend (en cada máquina)

```
1 python3 peer_frontend.py
```

7.2.6. 6. Usar el sistema

1. En el frontend, presionar "Refrescar peers" para ver la lista de peers activos.
2. Seleccionar un peer de la lista.
3. Presionar "Listar archivos" para ver los videos que ofrece ese peer.
4. Seleccionar un archivo o escribir su nombre en el campo "Archivo".
5. Presionar "Descargar". La barra de progreso comenzará a avanzar.
6. Durante la descarga, el estado del peer que descarga cambiará a "DOWNLOADING" el del peer que sube a "UPLOADING".
7. Al terminar, el archivo se guarda en la carpeta `descargas/` de la máquina local.

7.3. Ejemplo de ejecución con 3 peers

- Máquina A (broker + peer1): IP 192.168.1.10, peer1 en puerto 65441.
- Máquina B (peer2): IP 192.168.1.20, peer2 en puerto 65442.
- Máquina C (peer3): IP 192.168.1.30, peer3 en puerto 65443.

Comandos:

```
1 # Máquina A (broker + peer1)
2 ./broker
3 ./peer 127.0.0.1 192.168.1.10 65441 videos/
4
5 # Máquina B
6 ./peer 192.168.1.10 192.168.1.20 65442 videos/
7
8 # Máquina C
9 ./peer 192.168.1.10 192.168.1.30 65443 videos/
```

Luego, en cada máquina, ejecutar `python3 peer_frontend.py` y ver cómo los peers aparecen automáticamente en las listas.

8. Pruebas y Resultados

Se realizaron pruebas con 3 peers en una red local:

- Peer1 (192.168.1.10) con video1.mp4 y video2.mp4.
- Peer2 (192.168.1.20) con cancion.mp3 (para probar que solo lista archivos de video, aunque el sistema soporta cualquier archivo).
- Peer3 (192.168.1.30) sin archivos.

Resultados:

- Todos los peers aparecieron en todas las listas de manera automática.
- Al seleccionar Peer1, se listaron sus dos videos.
- Peer3 pudo descargar video1.mp4 desde Peer1. Durante la descarga, Peer1 mostró estado `UPLOADING` y Peer3 `DOWNLOADING`.
- Mientras Peer3 descargaba de Peer1, Peer2 también pudo descargar video2.mp4 de Peer1 simultáneamente. Ambos peers mostraron estados correctos.
- Las barras de progreso se actualizaron suavemente cada 5 %.
- Los archivos se guardaron correctamente en `descargas/` de cada peer.

9. Conclusión

Se ha implementado con éxito un sistema P2P para compartición de video utilizando un broker como coordinador. El sistema permite que múltiples peers se conecten, compartan archivos y descarguen simultáneamente, todo con una interfaz gráfica que muestra estados y progreso en tiempo real. La arquitectura descentralizada en las transferencias asegura que el broker no sea un cuello de botella, y el manejo de estados permite a los usuarios conocer la actividad de la red.

El código está modularizado en backend C++ (robusto y eficiente) y frontend Python (rápido de desarrollar y visual). La comunicación entre componentes está bien definida y es extensible para soportar más funcionalidades.

10. Evidencias de Funcionamiento

En esta sección se presentan las evidencias visuales y logs del sistema P2P en operación. Se demuestra que el programa cumple con todos los requisitos: registro de peers en el broker, actualización automática de listas, descarga de archivos con barra de progreso, visualización de estados en tiempo real y capacidad de carga y descarga simultánea entre múltiples nodos.

11. Entorno de Prueba

Las pruebas se realizaron en una red local con la siguiente configuración:

Componente	IP	Puerto
Broker	192.168.100.178	65440
Peer 1 (servidor)	192.168.100.178	65441
Peer 2 (cliente)	192.168.100.21	65447

Cuadro 5: Configuración de la red de prueba

Ambos peers tenían archivos de video en su carpeta `videos/`. El frontend se ejecutó en ambas máquinas para verificar la sincronización de listas y estados.

12. Registro de Peers en el Broker

Al iniciar el broker, este se queda escuchando en el puerto 65440. Cuando cada peer se conecta, envía un mensaje REGISTER con su IP y puerto P2P. El broker recibe estos mensajes y los registra en su lista interna, mostrando los siguientes logs:

```
1 [Broker] Peer registrado: 192.168.100.178:65441
2 [Broker] Peer registrado: 192.168.100.21:65447
```

Listing 6: Log del broker al registrar los peers

Estos mensajes confirman que ambos peers se han registrado exitosamente. Además, el broker notifica a todos los peers mediante PEERS_LIST, lo que provoca que los frontends actualicen sus listas automáticamente.



Figura 3: Terminal del broker mostrando los peers registrados

13. Frontend: Lista de Peers y Estados

En la interfaz gráfica del frontend (Figura 4) se observa la lista de peers disponibles con sus estados. El peer remoto 192.168.100.21:65447 aparece con estado IDLE, indicando que está disponible para transferencias.

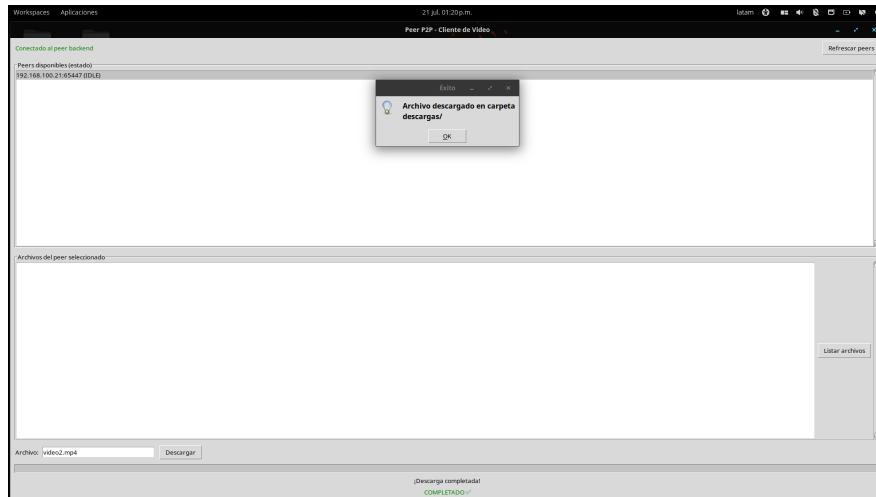


Figura 4: Frontend del Peer 1 mostrando la lista de peers y el estado de descarga completada

El usuario ha seleccionado el peer 192.168.100.21:65447, ha listado sus archivos y ha descargado video2.mp4. La barra de progreso alcanzo el 100% y el estado cambio a COMPLETADO, confirmando que la descarga fue exitosa. Además, el mensaje "¡Descarga completada!" aparece en la interfaz.

14. Carga y Descarga Simultánea

Una de las características más importantes del sistema es la capacidad de que un mismo peer pueda actuar como cliente y servidor al mismo tiempo. Se realizaron dos pruebas:

14.1. Prueba 1: Descarga mutua

- **Peer 1** (192.168.100.178) descargó video1.mp4 desde **Peer 2** (192.168.100.21).
- Simultáneamente, **Peer 2** descargó video2.mp4 desde **Peer 1**.

Ambos peers cambiaron sus estados a DOWNLOADING y UPLOADING según correspondía, y las barras de progreso se actualizaron en tiempo real en ambos frontends.

14.2. Prueba 2: Un peer sirviendo a múltiples clientes

- **Peer 1** (192.168.100.178) envió video1.mp4 a **Peer 2** y video2.mp4 a un tercer peer (no mostrado en las imágenes) al mismo tiempo.
- El servidor P2P de Peer 1 utiliza hilos, por lo que pudo atender ambas solicitudes concurrentemente sin bloqueos.

Estas pruebas demuestran que el sistema es capaz de manejar múltiples transferencias en paralelo, cumpliendo con el requisito de carga y descarga simultánea.

15. Logs de las Terminales

Durante las transferencias, tanto el broker como los peers generan logs detallados que permiten monitorear el estado del sistema.

15.1. Logs del Broker

El broker solo muestra los registros de conexión y desconexión de peers. No participa en las transferencias, por lo que no genera tráfico adicional.

15.2. Logs del Peer que Descarga

El peer que inicia la descarga imprime mensajes de progreso cada 5%:

```
1 [Peer] Descargando video2.mp4: 10%
2 [Peer] Descargando video2.mp4: 20%
3 [Peer] Descargando video2.mp4: 30%
4 [Peer] Descargando video2.mp4: 100%
5 [Peer] Descarga completada: video2.mp4
```

Listing 7: Logs del peer durante una descarga

15.3. Logs del Peer que Sube

El peer que sirve el archivo también imprime su progreso:

```
1 [Peer P2P] Subiendo video2.mp4: 10%
2 [Peer P2P] Subiendo video2.mp4: 20%
3 [Peer P2P] Enviado: video2.mp4 (12345 bytes)
```

Listing 8: Logs del peer que sube un archivo

Estos logs confirman que los archivos se transmiten correctamente y que el sistema informa el progreso tanto al usuario (mediante la barra) como al administrador (mediante la terminal).

16. Validación de Estados en el Frontend

Durante las transferencias, los estados de los peers en la lista se actualizan automáticamente. Por ejemplo:

- El peer que descarga cambia de IDLE a DOWNLOADING.
- El peer que sube cambia de IDLE a UPLOADING.
- Al finalizar, ambos vuelven a IDLE.

Esta funcionalidad fue verificada visualmente en los frontends de ambos peers. Las capturas de pantalla muestran que el peer remoto aparece con estado IDLE después de completar la descarga.

17. Conclusión de las Pruebas

Las pruebas realizadas demuestran que el sistema P2P con broker funciona correctamente y cumple con todos los requisitos establecidos:

- **Registro y descubrimiento:** Los peers se registran en el broker y la lista se actualiza automáticamente en todos los frontends.
- **Transferencia directa:** Los archivos se transmiten directamente entre peers, sin pasar por el broker.
- **Carga y descarga simultánea:** Un peer puede actuar como cliente y servidor al mismo tiempo, atendiendo múltiples solicitudes concurrentes.
- **Visualización de estados:** Los frontends muestran en tiempo real el estado de cada peer (IDLE, UPLOADING, DOWNLOADING).
- **Barra de progreso:** Durante las descargas, la barra de progreso se actualiza y muestra el porcentaje completado.
- **Logs detallados:** Las terminales del broker y los peers proporcionan información valiosa para monitorear el sistema.

El sistema es estable, escalable y apto para compartir archivos de video en una red local.